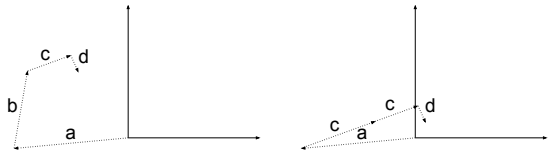

Figure 3: The intervention described by Equations 10–13 where the input sequence is (a, b, c, d) and the intervention is to fix the second position to be the token c.

encodes how the model represents that token in its hidden state vector. To replace a token in a sequence $i_1 \dots i_L$, we add the difference of the embeddings for a new $\hat{i}_j$ and old i_j token scaled according to the target position j. Our goal is to intervene upon the hidden representation $\hat{\boldsymbol{h}}_L$ so that the sequence decoded is $i_1 \dots \hat{i}_j \dots i_L$. We randomly sample $\hat{i}_j$ and use inputs from the GRU training data.

7.2 Method: Onion Interventions

To replace token i_j with token $\hat{i}_j$, we add the difference of the corresponding token embeddings scaled by a factor determined by the position j. We parameterize this as:

$$\boldsymbol{x} = \boldsymbol{E}[i_j] \tag{10}$$

$$\hat{\boldsymbol{x}} = \boldsymbol{E}[\hat{i}_j] \tag{11}$$

$$\boldsymbol{s} = \boldsymbol{g}\boldsymbol{\gamma}^j + \boldsymbol{\beta}j + \boldsymbol{b} \tag{12}$$

$$\boldsymbol{h}' = \hat{\boldsymbol{h}} + \boldsymbol{s} \odot (\hat{\boldsymbol{x}} - \boldsymbol{x}) \tag{13}$$

where $\boldsymbol{E} \in \mathbb{R}^{N_S \times N}$ is the embedding for the tokens (distinct from the the GRU input embedding, learned from scratch for the intervention), and $\boldsymbol{g}, \boldsymbol{\gamma}, \boldsymbol{\beta}, \boldsymbol{b} \in \mathbb{R}^N$ are learned scaling parameters. Intuitively, $\boldsymbol{s}$ is the scale used for the token in position j. Its main component is the exponential term $\boldsymbol{\gamma}$. In order to replace the token in the sequence, compute the difference of their embeddings, and scale them to the scale corresponding to the given position. Different channels in the state $\boldsymbol{h} \in \mathbb{R}^N$ might have different scales. Figure 3 depicts an example intervention, extending figure 1.

7.3 Results

The last row of Table 2 shows that our onion intervention achieves significantly better accuracy on the small models compared to the alternative unigram and bigram interventions. For example, for $N = 64$, the onion intervention achieves 87% accuracy compared to the 1% of the bigram intervention. As a control, if we fix $\boldsymbol{\gamma} = 1$ and $\boldsymbol{\beta} = 1$, we only reach 21% accuracy.

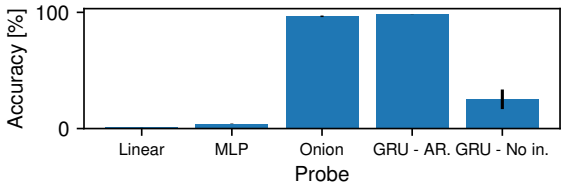

Figure 4: Accuracy of different probes on the final representation $\boldsymbol{h}_L$ of GRUs with $N = 64$ and autoregressive input (mean of 5 runs; $\pm$ 1 s.d.). Only the probes that use autoregressive denoising can successfully decode the sequence.

7.4 Discussion

Why do GRUs learn onion representations? In order to distinguish N_S tokens stored in L_{max} possible positions, the model needs to be able to distinguish between $N_S \times L_{\text{max}}$ different directions in the feature space. In our experiments this is 300 possible directions, stored in a 64-dimensional vector space. In contrast, for onion representations, they only have to distinguish between $N_S = 30$ directions at different orders of magnitude.

Onion representations require unpeeling via autoregression. We train a variety of probes to decode the final representation $\boldsymbol{h}_L$ after encoding the input sequence of GRUs with $N = 64$, which learn onion representation. We show our results in figure 4. The *linear* and *MLP* probes predict the entire sequence at once by mapping the hidden vector $\boldsymbol{h}_L \in \mathbb{R}^N$ to the logits for each timestep $\boldsymbol{y}_{\text{all}} \in \mathbb{R}^{N_S \times L_{\text{max}}}$. The *GRU Autoregressive (GRU – AR)* probe is equivalent to the original model, and we use it as a check to verify that the decoding is easy to learn. The *GRU – No input* probe is similar, but unlike the original decoder of the model, it does not receive an autoregressive input.

The probe results confirm that it's not merely a free choice whether the decoder uses an autoregressive input or not: if an onion representation is learned during the training phase, it is impossible to decode it with a non-autoregressive decoder, contrary to the same-size models that are trained without an autoregressive input, shown in Table 4 in Appendix E.3. We also show the special probe we designed for onion representations in a similar spirit to the intervention described in section 7.2, which performs almost perfectly. More details can be found in Appendix E.3.

What is the feature space of an onion representation? Together, the embeddings $\boldsymbol{E}$ learned for each token and the probe $\mathcal{P}$ that predicts the to-

ken sequence form an encoder $\mathcal{F}$ that projects the hidden vector $\boldsymbol{h}_L$ into a new feature space:

$$\mathcal{F}(\boldsymbol{h}_L) = \langle \boldsymbol{E}[\mathcal{P}(\boldsymbol{h}_L)_1], \ldots, \boldsymbol{E}[\mathcal{P}(\boldsymbol{h}_L)_L], \boldsymbol{h}_L - \sum_{j=2}^{L} \boldsymbol{E}[\mathcal{P}(\boldsymbol{h}_L)_j] \cdot \boldsymbol{s}_j \rangle$$

where the first L features are the token embeddings corresponding to the token sequence predicted by the probe and the final feature is what remains of the hidden state after those embeddings are removed. The inverse is a simple weighted sum:

$$\mathcal{F}^{-1}(\mathbf{f}) = \mathbf{f}_{L+1} + \sum_{j=1}^{L} \mathbf{f}_j \cdot \boldsymbol{s}_j$$

If the probe had perfect accuracy, this inverse would be perfect. Since our probe has 98% accuracy, there is a reconstruction loss when applying the featurizer and inverse featurizer (similar to sparse autoencoders, e.g., Bricken et al. 2023; Huben et al. 2024).

This onion feature space is parameterized by an embedding for each token, a dynamic scaling factor, and a probe. In contrast, a single linear feature is just a vector. However, because $\mathcal{F}$ is (approximately) bijective, we know that $\mathcal{F}$ (approximately) induces an intervention algebra (Geiger et al., 2024a) where each feature is modular and can be intervened upon separately from other features.

Our embedding-based interventions are equivalent to onion interchange interventions. We evaluated the linear representations of large networks with interchange interventions that fixed a linear subspace to the value it would have taken on if a different token sequence were input to the model. There is a corresponding interchange intervention for onion representations. However, it turns out that these onion interchange interventions are equivalent to the scaled difference of embeddings used in our experiments (see Appendix B).

Why do Onion interventions also work on large models? Surprisingly, the onion intervention works well on the big models that have linear representations of position ($N \geq 256$). We hypothesize that this is possible because all of the models start with gates open before closing them in a monotonic, sequential manner as the input sequence is processed. This enables the scaling-based onion intervention to simulate the actual gating pattern sufficiently closely to be able to perform the intervention well enough. The intervention cannot express arbitrarily sharp gate transitions but can compensate for them by creating an ensemble with different decay factors for the different channels.

From Table 5 in the Appendix, it can be seen that the onion intervention achieves significantly worse performance on the small non-autoregressive models that use linear representations compared to the autoregressive ones. This is expected, as the onion intervention cannot express an arbitrary gating pattern that might be learned by these models.

8 Discussion and Conclusion

The preceding experiments show that GRUs learn highly structured and systematic solutions to the repeat task. It should not be overlooked that two of these solutions (those based in unigram and bigram subspaces) are consistent with the general guiding intuitions behind the LRH and so help to illustrate the value of testing hypotheses in that space. However, our primary goal is to highlight the onion solution, as it falls outside the LRH.

Our hope is that this spurs researchers working on mechanistic interpretability to consider a wider range of techniques. The field is rapidly converging around methods that can only find solutions consistent with the LRH, as we briefly reviewed in section 2. In this context, counterexamples to the LRH have significant empirical and theoretical value, as Olah (2024) makes clear:

> But if representations are not mathematically linear in the sense described above [in a definition of the LRH], it's back to the drawing board – a huge number of questions like "how should we think about weights?" are reopened.

Our counterexample is on a small network, but our task is also very simple. Very large networks solving very complex tasks may also find solutions that fall outside of the LRH.

There is also a methodological lesson behind our counterexample to the LRH. Much interpretability work is guided by concerns related to AI safety. The reasoning here is that we need to deeply understand models if we are going to be able to certify them as safe and robust, and detect unsafe mechanisms and behaviors before they cause real harm. Given such goals, it is essential that we analyze these models in an unbiased and open-minded way.

9 Limitations

The generality of onion representations. Onion representations are well fit for memorizing a sequence in order or in reverse order, but they cannot provide a general storage mechanism with arbitrary access patterns. It is unclear if such representations are useful in models trained on more complex real-world tasks.

Using GRU models. Our exploration is limited to GRU models, which themselves might have less interest in the current Transformer-dominated state of the field. However, we suspect that the same representations are beneficial for other gated RNNs as well, such as LSTMs. Although we have a reason to believe that such representations can emerge in Transformers and state space models as well, we do not verify this hypothesis empirically.

Onion representations only emerge in small models. This might indicate that onion representations are not a problem for bigger models used in practice. However, this might not be the case: LLMs, which are much bigger, operate on an enormous feature space using a relatively small residual stream. Thus, the pressure to compress representations and the potential for similar representations to emerge could be well motivated there as well.

Numerical precision. The number of elements that can be stored in onion representations depends on the numerical precision of the data type used for the activations. We found that the network finds it easy to use these representations even with 16-bit floating point precision (bf16), potentially because multiple redundant channels of the state can be used as an ensemble. It remains unclear what the capacity of such representations is.

10 Acknowledgements

Christopher D. Manning is a CIFAR Fellow. This research is in part supported by a grant from Open Philanthropy.

References

Sanjeev Arora, Yuanzhi Li, Yingyu Liang, Tengyu Ma, and Andrej Risteski. 2016. A latent variable model approach to PMI-based word embeddings. *Transactions of the Association for Computational Linguistics*, 4:385–399.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *Preprint arXiv:1607.06450*.

Nora Belrose, David Schneider-Joseph, Shauli Ravfogel, Ryan Cotterell, Edward Raff, and Stella Biderman. 2023. LEACE: perfect linear concept erasure in closed form. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *Preprint arXiv:1308.3432*.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.

Róbert Csordás, Sjoerd van Steenkiste, and Jürgen Schmidhuber. 2021. Are neural nets modular? Inspecting functional modularity through differentiable weight masks. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*.

Xander Davies, Max Nadeau, Nikhil Prakash, Tamar Rott Shaham, and David Bau. 2023. Discovering variable binding circuitry with desiderata. *CoRR*, abs/2307.03637.

Nicola De Cao, Michael Sejr Schlichtkrull, Wilker Aziz, and Ivan Titov. 2020. How do decisions emerge across layers in neural models? interpretation with differentiable masking. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3243–3255, Online. Association for Computational Linguistics.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and